```
  update(element, newPriority)
}
```

These above are just the public methods defining the API of our class.

First things first: we will assume, hereinafter, that we store all the (element, priority) tuples added to the heap in an array named `pairs`.

Listing 2.1 The DHeap class properties

```
class DHeap
  #type Array[Pair]
  pairs
  function DHeap(pairs=[])
```

If we have a variable `p` holding such a tuple, we embrace a specific syntax in our pseudo-code for destructured assignment of its fields into two variables, as shown in appendix A:

```
(element, priority) ← p
```

> We also assume that the tuple's fields are named, so that we can access, in turn, `p.element` and `p.priority`, or create a tuple `p` with this syntax:

```
p ← (element='x', priority=1)
```